



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 02

**NOMBRE COMPLETO: NOYOLA TORRES PABLO
SEBASTIAN**

N° de Cuenta: 320023213

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

**FECHA DE ENTREGA LÍMITE: 05 DE SEPTIEMBRE DE
2026**

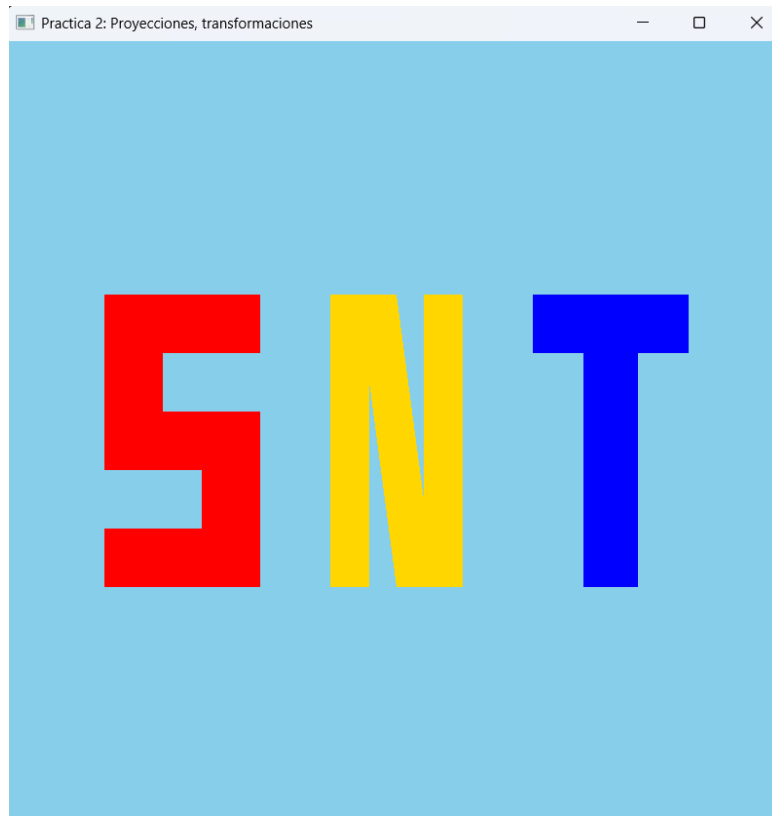
CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1.- Dibujar las iniciales de sus nombres cada letra de un color diferente.

Capturas de ejecución:



Capturas de código:

```

110 void CrearLetrasyFiguras() {
111
112     GLfloat vertices_letras[] = {
113         //PARA INICIALES SNT
114         // PARA S - Color Rojo: 1.0f, 0.0f, 0.0f
115         // Barra superior
116         -0.75f, 0.35f, 0.0f, 1.0f, 0.0f, 0.0f, -0.35f, 0.35f, 0.0f, 1.0f, 0.0f, 0.0f, -0.75f, 0.20f, 0.0f, 1.0f, 0.0f, 0.0f,
117         -0.35f, 0.35f, 0.0f, 1.0f, 0.0f, 0.0f, -0.35f, 0.20f, 0.0f, 1.0f, 0.0f, 0.0f, -0.75f, 0.20f, 0.0f, 1.0f, 0.0f, 0.0f,
118         // Bajada superior izquierda
119         -0.75f, 0.20f, 0.0f, 1.0f, 0.0f, 0.0f, -0.60f, 0.20f, 0.0f, 1.0f, 0.0f, 0.0f, -0.75f, 0.05f, 0.0f, 1.0f, 0.0f, 0.0f,
120         -0.60f, 0.20f, 0.0f, 1.0f, 0.0f, 0.0f, -0.60f, 0.05f, 0.0f, 1.0f, 0.0f, 0.0f, -0.75f, 0.05f, 0.0f, 1.0f, 0.0f, 0.0f,
121         // Barra central
122         -0.75f, 0.05f, 0.0f, 1.0f, 0.0f, 0.0f, -0.35f, 0.05f, 0.0f, 1.0f, 0.0f, 0.0f, -0.75f, -0.10f, 0.0f, 1.0f, 0.0f, 0.0f,
123         -0.35f, 0.05f, 0.0f, 1.0f, 0.0f, 0.0f, -0.35f, -0.10f, 0.0f, 1.0f, 0.0f, 0.0f, -0.75f, -0.10f, 0.0f, 1.0f, 0.0f, 0.0f,
124         // Bajada inferior derecha
125         -0.50f, -0.10f, 0.0f, 1.0f, 0.0f, 0.0f, -0.35f, -0.10f, 0.0f, 1.0f, 0.0f, 0.0f, -0.50f, -0.25f, 0.0f, 1.0f, 0.0f, 0.0f,
126         -0.35f, -0.10f, 0.0f, 1.0f, 0.0f, 0.0f, -0.35f, -0.25f, 0.0f, 1.0f, 0.0f, 0.0f, -0.50f, -0.25f, 0.0f, 1.0f, 0.0f, 0.0f,
127         // Barra inferior
128         -0.75f, -0.25f, 0.0f, 1.0f, 0.0f, 0.0f, -0.35f, -0.25f, 0.0f, 1.0f, 0.0f, 0.0f, -0.75f, -0.40f, 0.0f, 1.0f, 0.0f, 0.0f,
129         -0.35f, -0.25f, 0.0f, 1.0f, 0.0f, 0.0f, -0.35f, -0.40f, 0.0f, 1.0f, 0.0f, 0.0f, -0.75f, -0.40f, 0.0f, 1.0f, 0.0f, 0.0f,
130
131         // PARA N - Color Amarillo Dorado: 1.0f, 0.84f, 0.0f
132         // Barra izquierda
133         -0.17f, 0.35f, 0.0f, 1.0f, 0.84f, 0.0f, -0.07f, 0.35f, 0.0f, 1.0f, 0.84f, 0.0f, -0.17f, -0.40f, 0.0f, 1.0f, 0.84f, 0.0f,
134         -0.07f, 0.35f, 0.0f, 1.0f, 0.84f, 0.0f, -0.07f, -0.40f, 0.0f, 1.0f, 0.84f, 0.0f, -0.17f, -0.40f, 0.0f, 1.0f, 0.84f, 0.0f,
135         // Barra diagonal del centro
136         -0.10f, 0.35f, 0.0f, 1.0f, 0.84f, 0.0f, 0.10f, -0.40f, 0.0f, 1.0f, 0.84f, 0.0f, 0.00f, 0.35f, 0.0f, 1.0f, 0.84f, 0.0f,
137         0.10f, -0.40f, 0.0f, 1.0f, 0.84f, 0.0f, -0.10f, 0.35f, 0.0f, 1.0f, 0.84f, 0.0f, 0.00f, -0.40f, 0.0f, 1.0f, 0.84f, 0.0f,
138         // Barra derecha
139         0.07f, 0.35f, 0.0f, 1.0f, 0.84f, 0.0f, 0.17f, 0.35f, 0.0f, 1.0f, 0.84f, 0.0f, 0.07f, -0.40f, 0.0f, 1.0f, 0.84f, 0.0f,
140         0.17f, 0.35f, 0.0f, 1.0f, 0.84f, 0.0f, 0.17f, -0.40f, 0.0f, 1.0f, 0.84f, 0.0f, 0.07f, -0.40f, 0.0f, 1.0f, 0.84f, 0.0f,
141
142         // PARA T - Color Azul: 0.0f, 0.0f, 1.0f
143         // Barra horizontal superior
144         0.35f, 0.35f, 0.0f, 0.0f, 0.0f, 1.0f, 0.75f, 0.35f, 0.0f, 0.0f, 0.0f, 1.0f, 0.35f, 0.20f, 0.0f, 0.0f, 0.0f, 1.0f,
145         0.75f, 0.35f, 0.0f, 0.0f, 0.0f, 1.0f, 0.75f, 0.20f, 0.0f, 0.0f, 0.0f, 1.0f, 0.35f, 0.20f, 0.0f, 0.0f, 0.0f, 1.0f,
146         // Barra vertical central
147
148         // Barra vertical central
149         0.48f, 0.20f, 0.0f, 0.0f, 0.0f, 1.0f, 0.62f, 0.20f, 0.0f, 0.0f, 0.0f, 1.0f, 0.48f, -0.40f, 0.0f, 0.0f, 0.0f, 1.0f,
150         0.62f, 0.20f, 0.0f, 0.0f, 0.0f, 1.0f, 0.62f, -0.40f, 0.0f, 0.0f, 0.0f, 1.0f, 0.48f, -0.40f, 0.0f, 0.0f, 0.0f, 1.0f
151     };
152     MeshColor* letras = new MeshColor();
153     letras->CreateMeshColor(vertices_letras, 360);
154     meshColorList.push_back(letras);
155
156
157     // PARA LETRAS DE INICIALES CON COLOR DIFERENTE CADA UNA
158     shaderList[1].useShader();
159     uniformModel = shaderList[1].getModelLocation();
160     uniformProjection = shaderList[1].getProjectLocation();
161     model = glm::mat4(1.0);
162     model = glm::translate(model, glm::vec3(0.0f, 0.0f, -2.0f));
163     model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
164     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
165     glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
166     meshColorList[0]->RenderMeshColor();
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

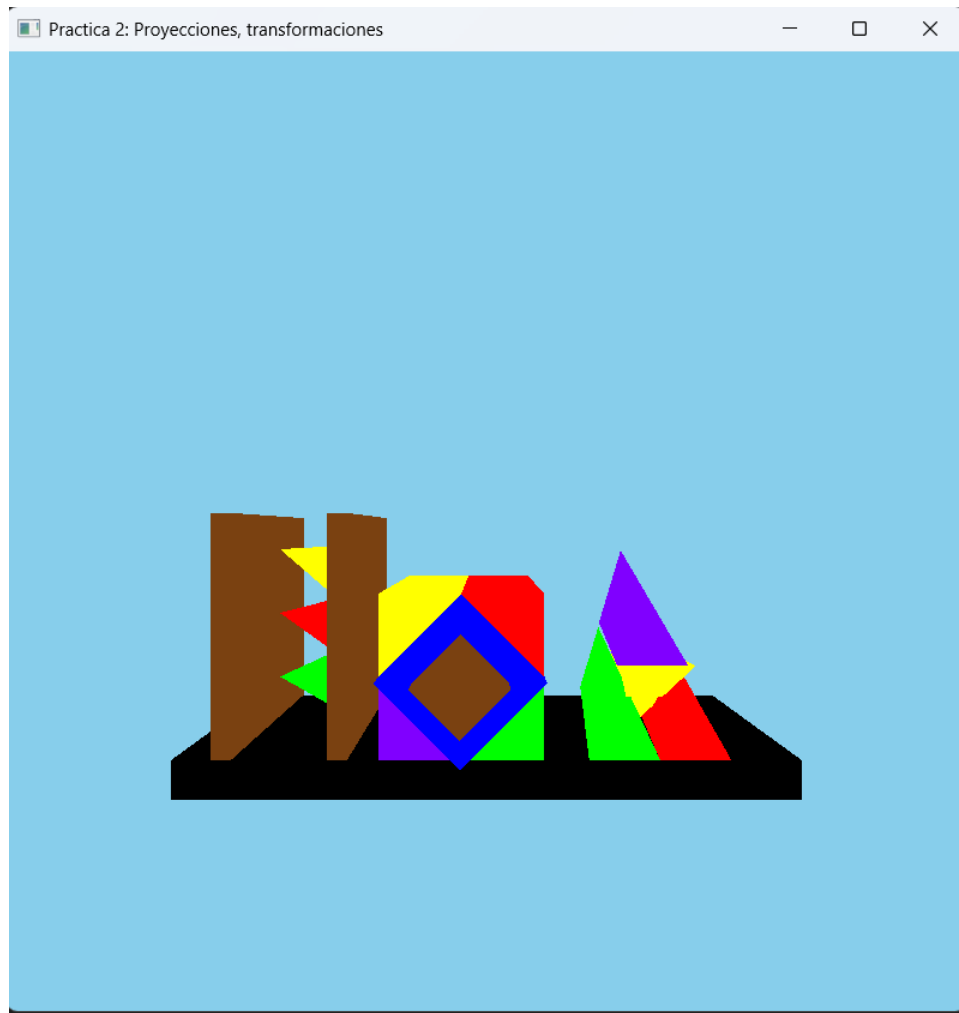
```

Para la realización de este ejercicio, que era implementar la lógica en OpenGL para definir, almacenar y renderizar las iniciales de mi nombre (SNT) con colores distintos por letra, aproveche el arreglo de vértices de letras utilizados en la práctica anterior, añadiéndoles las coordenadas de color por cada vértice, los cuales son cargados en la memoria de la tarjeta gráfica a través de un objeto MeshColor y añadidos a meshColorList. Posteriormente, en la sección del main dentro del while, se activa el shader correspondiente, se configuran las matrices uniformes de modelo y proyección con sus respectivas transformaciones de traslación y escala, y se manda a dibujar el objeto en pantalla ejecutando el método RenderMeshColor().

2.- Generar el dibujo de las 3 figuras de clase, pero en lugar de instanciar triángulos y cuadrados será instanciando pirámides y cubos, para esto se requiere crear shaders diferentes de los colores: verde, azul, café y magente

adicionales al shaderrojo. Se debe de entregar los archivos de shader diferentes dentro de un zip adicional al main y documento escrito.

Capturas de ejecución:



Capturas de código:

```

22 //Vertex Shader
23 static const char* vShader = "shaders/shader.vert";
24 static const char* fShader = "shaders/shader.frag";
25 static const char* vShaderColor = "shaders/shadercolor.vert";
26 static const char* fShaderColor = "shaders/shadercolor.frag";
27
28 //shaders nuevos se crearían acá
29 static const char* vShaderRojo = "shaders/shaderrojo.vert";
30 static const char* fShaderRojo = "shaders/shaderrojo.frag";
31 static const char* vShaderAzul = "shaders/shaderazul.vert";
32 static const char* fShaderAzul = "shaders/shaderazul.frag";
33 static const char* vShaderVerde = "shaders/shaderverde.vert";
34 static const char* fShaderVerde = "shaders/shaderverde.frag";
35 static const char* vShaderMagenta = "shaders/shadermagenta.vert";
36 static const char* fShaderMagenta = "shaders/shadermagenta.frag";
37 static const char* vShaderCafe = "shaders/shadercafe.vert";
38 static const char* fShaderCafe = "shaders/shadercafe.frag";
39 static const char* vShaderNegro = "shaders/shadernegro.vert";
40 static const char* fShaderNegro = "shaders/shadernegro.frag";
41 static const char* vShaderAmarillo = "shaders/shaderamarillo.vert";
42 static const char* fShaderAmarillo = "shaders/shaderamarillo.frag";
43

```

```

312 void CreateShaders()
313 {
314
315     Shader* shader0 = new Shader(); //shader para usar índices: objetos: cubo y pirámide
316     shader0->CreateFromFiles(vShader, fShader);
317     shaderList.push_back(*shader0);
318
319     Shader* shader1 = new Shader(); //shader para usar color como parte del VAO: letras
320     shader1->CreateFromFiles(vShaderColor, fShaderColor);
321     shaderList.push_back(*shader1);
322
323     Shader* shader2 = new Shader();
324     shader2->CreateFromFiles(vShaderRojo, fShaderRojo);
325     shaderList.push_back(*shader2);
326
327     Shader* shader3 = new Shader();
328     shader3->CreateFromFiles(vShaderAzul, fShaderAzul);
329     shaderList.push_back(*shader3);
330
331     Shader* shader4 = new Shader();
332     shader4->CreateFromFiles(vShaderVerde, fShaderVerde);
333     shaderList.push_back(*shader4);
334
335     Shader* shader5 = new Shader();
336     shader5->CreateFromFiles(vShaderCafe, fShaderCafe);
337     shaderList.push_back(*shader5);
338
339     Shader* shader6 = new Shader();
340     shader6->CreateFromFiles(vShaderAmarillo, fShaderAmarillo);
341     shaderList.push_back(*shader6);
342
343     Shader* shader7 = new Shader();
344     shader7->CreateFromFiles(vShaderMagenta, fShaderMagenta);
345     shaderList.push_back(*shader7);
346
347     Shader* shader8 = new Shader();

```

```

347     Shader* shader8 = new Shader();
348     shader8->CreateFromFiles(vShaderNegro, fShaderNegro);
349     shaderList.push_back(*shader8);
350 }

```

```

362 //Projection: Matriz de Dimensión 4x4 para indicar si vemos en 2D(orthogonal) o en 3D perspectiva
363 //glm::mat4 projection = glm::ortho(-1.0f, 1.0f, -1.0f, 1.0f, 0.1f, 100.0f);
364 glm::mat4 projection = glm::perspective(glm::radians(60.0f), mainWindow.getBufferWidth() / mainWindow.getBufferHeight(), 0.1f, 100.0f);

```

```

391 // PARA FIGURAS DE IMAGEN DE CLASE
392 // Para barra negra rectangular de base
393 shaderList[8].useShader();
394 uniformModel = shaderList[8].getModelLocation();
395 uniformProjection = shaderList[8].getProjectLocation();
396 model = glm::mat4(1.0f);
397 model = glm::translate(model, glm::vec3(0.0f, -0.75f, -3.0f));
398 model = glm::scale(model, glm::vec3(1.9f, 0.12f, 1.0f));
399 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
400 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
401 meshList[1]->RenderMesh();
402
403 // Para primer figura
404 // Poste izquierdo cafe
405 shaderList[5].useShader();
406 uniformModel = shaderList[5].getModelLocation();
407 uniformProjection = shaderList[5].getProjectLocation();
408 model = glm::mat4(1.0f);
409 model = glm::translate(model, glm::vec3(-0.8f, -0.32f, -3.0f));
410 model = glm::scale(model, glm::vec3(0.06f, 0.75f, 1.0f));
411 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
412 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
413 meshList[1]->RenderMesh();
414
415 // Poste derecho cafe
416 shaderList[5].useShader();
417 uniformModel = shaderList[5].getModelLocation();
418 uniformProjection = shaderList[5].getProjectLocation();
419 model = glm::mat4(1.0f);
420 model = glm::translate(model, glm::vec3(-0.45f, -0.32f, -3.0f));
421 model = glm::scale(model, glm::vec3(0.06f, 0.75f, 1.0f));
422 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
423 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));

```

```

423 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
424 meshList[1]->RenderMesh();
425
426 // Para triangulo verde inferior invertido
427 shaderList[4].useShader();
428 uniformModel = shaderList[4].getModelLocation();
429 uniformProjection = shaderList[4].getProjectLocation();
430 model = glm::mat4(1.0f);
431 model = glm::translate(model, glm::vec3(-0.625f, -0.63f, -3.0f)); // Un poco más abajo
432 model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
433 model = glm::scale(model, glm::vec3(0.24f, 0.21f, 1.8f)); // Ligeramente más compactos
434 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
435 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
436 meshList[0]->RenderMesh();
437
438 // Para triangulo rojo medio invertido
439 shaderList[2].useShader();
440 uniformModel = shaderList[2].getModelLocation();
441 uniformProjection = shaderList[2].getProjectLocation();
442 model = glm::mat4(1.0f);
443 model = glm::translate(model, glm::vec3(-0.625f, -0.40f, -3.0f));
444 model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
445 model = glm::scale(model, glm::vec3(0.24f, 0.21f, 1.8f));
446 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
447 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
448 meshList[0]->RenderMesh();
449
450 // Para triangulo amarillo superior invertido
451 shaderList[6].useShader();
452 uniformModel = shaderList[6].getModelLocation();
453 uniformProjection = shaderList[6].getProjectLocation();
454 model = glm::mat4(1.0f);
455 model = glm::translate(model, glm::vec3(-0.625f, -0.17f, -3.0f)); // Mayor espacio libre arriba en los postes
456 model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
457 model = glm::scale(model, glm::vec3(0.24f, 0.21f, 1.8f));
458 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
459 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
460 meshList[0]->RenderMesh();

```

```

462 // Para segunda figura
463 // Para cubo morado (Magente)
464 shaderList[7].useShader();
465 uniformModel = shaderList[7].getModelLocation();
466 uniformProjection = shaderList[7].getProjectLocation();
467 model = glm::mat4(1.0f);
468 model = glm::translate(model, glm::vec3(-0.2f, -0.564f, -3.0f));
469 model = glm::scale(model, glm::vec3(0.25f, 0.25f, 1.0f));
470 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
471 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
472 meshList[1] -> RenderMesh();
473
474 // Para cubo verde
475 shaderList[4].useShader();
476 uniformModel = shaderList[4].getModelLocation();
477 uniformProjection = shaderList[4].getProjectLocation();
478 model = glm::mat4(1.0f);
479 model = glm::translate(model, glm::vec3(0.05f, -0.564f, -3.0f));
480 model = glm::scale(model, glm::vec3(0.25f, 0.25f, 1.0f));
481 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
482 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
483 meshList[1] -> RenderMesh();
484
485 // Para cubo amarillo
486 shaderList[6].useShader();
487 uniformModel = shaderList[6].getModelLocation();
488 uniformProjection = shaderList[6].getProjectLocation();
489 model = glm::mat4(1.0f);
490 model = glm::translate(model, glm::vec3(-0.2f, -0.313f, -3.0f));
491 model = glm::scale(model, glm::vec3(0.25f, 0.25f, 1.0f));
492 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
493 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
494 meshList[1] -> RenderMesh();
495
496 // Para cubo rojo
497 shaderList[2].useShader();
498 uniformModel = shaderList[2].getModelLocation();
499 uniformProjection = shaderList[2].getProjectLocation();

```



```

498 uniformModel = shaderList[2].getModelLocation();
499 uniformProjection = shaderList[2].getProjectLocation();
500 model = glm::mat4(1.0f);
501 model = glm::translate(model, glm::vec3(0.05f, -0.313f, -3.0f));
502 model = glm::scale(model, glm::vec3(0.25f, 0.25f, 1.0f));
503 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
504 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
505 meshList[1]->RenderMesh();
506
507 // Para cubo azul
508 shaderList[3].useShader();
509 uniformModel = shaderList[3].getModelLocation();
510 uniformProjection = shaderList[3].getProjectLocation();
511 model = glm::mat4(1.0f);
512 model = glm::translate(model, glm::vec3(-.075f, -0.44f, -2.9f));
513 model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
514 model = glm::scale(model, glm::vec3(0.355f, 0.355f, 1.0f));
515 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
516 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
517 meshList[1]->RenderMesh();
518
519 // Para cubo Cafe
520 shaderList[5].useShader();
521 uniformModel = shaderList[5].getModelLocation();
522 uniformProjection = shaderList[5].getProjectLocation();
523 model = glm::mat4(1.0f);
524 model = glm::translate(model, glm::vec3(-.075f, -0.44f, -2.8f));
525 model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
526 model = glm::scale(model, glm::vec3(0.2f, 0.2f, 1.0f));
527 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
528 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
529 meshList[1]->RenderMesh();
530
531 // Para tercer figura
532 // Para piramide verde
533 shaderList[4].useShader();
534 uniformModel = shaderList[4].getModelLocation();
535 uniformProjection = shaderList[4].getProjectLocation();

```

```

532 // Para piramide verde
533 shaderList[4].useShader();
534 uniformModel = shaderList[4].getModelLocation();
535 uniformProjection = shaderList[4].getProjectLocation();
536 model = glm::mat4(1.0f);
537 model = glm::translate(model, glm::vec3(0.35f, -0.44f, -2.1f));
538 model = glm::scale(model, glm::vec3(0.18f, 0.28f, 2.0f)); // Z muy delgado para que sea plano
539 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
540 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
541 meshList[0]->RenderMesh();
542
543 // Para piramide roja
544 shaderList[2].useShader();
545 uniformModel = shaderList[2].getModelLocation();
546 uniformProjection = shaderList[2].getProjectLocation();
547 model = glm::mat4(1.0f);
548 model = glm::translate(model, glm::vec3(0.53f, -0.44f, -2.1f));
549 model = glm::scale(model, glm::vec3(0.18f, 0.28f, 2.0f));
550 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
551 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
552 meshList[0]->RenderMesh();
553
554 // Para piramide amarilla invertida
555 shaderList[6].useShader();
556 uniformModel = shaderList[6].getModelLocation();
557 uniformProjection = shaderList[6].getProjectLocation();
558 model = glm::mat4(1.0f);
559 model = glm::translate(model, glm::vec3(0.44f, -0.48f, -2.1f));
560 model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
561 model = glm::scale(model, glm::vec3(0.18f, 0.28f, 2.0f));
562 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
563 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
564 meshList[0]->RenderMesh();
565
566 // Para piramide morada (magenta)
567 shaderList[7].useShader();
568 uniformModel = shaderList[7].getModelLocation();
569 uniformProjection = shaderList[7].getProjectLocation();

```

```

550 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
551 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
552 meshList[0]->RenderMesh();
553
554 // Para piramide amarilla invertida
555 shaderList[6].useShader();
556 uniformModel = shaderList[6].getModelLocation();
557 uniformProjection = shaderList[6].getProjectLocation();
558 model = glm::mat4(1.0f);
559 model = glm::translate(model, glm::vec3(0.44f, -0.48f, -2.1f));
560 model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
561 model = glm::scale(model, glm::vec3(0.18f, 0.28f, 2.0f));
562 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
563 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
564 meshList[0]->RenderMesh();
565
566 // Para piramide morada (magenta)
567 shaderList[7].useShader();
568 uniformModel = shaderList[7].getModelLocation();
569 uniformProjection = shaderList[7].getProjectLocation();
570 model = glm::mat4(1.0f);
571 model = glm::translate(model, glm::vec3(0.42f, -0.2f, -2.1f));
572 model = glm::scale(model, glm::vec3(0.18f, 0.28f, 2.0f));
573 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
574 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
575 meshList[0]->RenderMesh();
576

```

En este ejercicio declaramos las rutas de los archivos de shaders independientes para cada color (rojo, azul, verde, magenta, café, negro y amarillo) y los cargamos dinámicamente en la función `CreateShaders()` para agregarlos al vector `shaderList`. Posteriormente, en el bucle principal de renderizado, utilizamos una proyección en perspectiva (`glm::perspective`) descomentada en lugar de la ortogonal (`glm::ortho`), ya que este ejercicio requiere visualizar las figuras en 3D volumétrico con profundidad y efecto de lejanía, a diferencia del ejercicio anterior donde se empleaba `glm::ortho` para trabajar de forma plana en 2D.

Finalmente, instanciamos las tres figuras geométricas de la clase utilizando cubos (`meshList[1]`) y pirámides (`meshList[0]`), aplicando transformaciones mediante `glm::translate`, `glm::scale` y `glm::rotate`, y asociando a cada elemento su respectivo shader de color para construir de manera correcta los postes, bases y estructuras tridimensionales de las figuras.

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Para esta práctica no se presentaron problemas en la realización del ejercicio 1; sin embargo, para el ejercicio 2 la situación fue distinta, ya que enfrentamos un gran reto al adaptar las figuras de la imagen de una perspectiva 2D a 3D. Tomé como base los trazos realizados en el ejercicio de clase los cuales empleaban una proyección; dibujar los cubos fue relativamente sencillo, pero en el caso de las pirámides el proceso resultó sumamente tardado y difícil. Al pasar a la proyección 3D, se deformaban demasiado y no lograban adoptar la forma de la imagen original, por lo que me tomó mucho tiempo ajustar las coordenadas de escala (`scale`) y traslación (`translation`) de cada figura. La única forma de resolverlo fue mediante prueba y error, modificando los valores y ejecutando el programa una y otra vez para observar cómo se visualizaban mejor.

Para las figuras 1 y 3, donde se empleaban pirámides, al no lograr percibirse adecuadamente, opté por hacer prismas triangulares para dar la apariencia deseada de frente. Aun así, personalmente no quedé del todo satisfecho con el resultado obtenido en la primera y tercera figura; sin embargo, decidí dejarlo así porque fue lo mejor que pude lograr y me tomó muchísimo tiempo, mucho más del que esperaba.

3.- Conclusión:

- a. Los ejercicios del reporte: Complejidad, Explicación.
- b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

c. Conclusión

Tras la realización de esta práctica, considero que los ejercicios fueron excelentes para poner en práctica los conceptos vistos en clase; personalmente, me permitieron comprenderlos y dominarlos mejor, además de que los temas se abordaron de forma correcta.

Por otro lado, al menos a mí sí me costó mucho trabajo realizar el segundo ejercicio, pues me tomó bastante tiempo. A pesar de que ya contaba con experiencia en este tipo de actividades, tardé mucho más de lo que creía; realmente no pensé que fuera a costarme tanto esfuerzo, por lo que requirió una gran inversión de tiempo.

Finalmente, considero que fue una práctica muy enriquecedora, pero espero administrar mejor mi tiempo y anticiparme a la dificultad que se pueda presentar en futuras entregas.

1. Bibliografía en formato APA

Khronos Group. (s.f.). OpenGL API Documentation. Recuperado de <https://www.khronos.org/opengl/>

de Vries, J. (2020). LearnOpenGL: Learn modern OpenGL graphics programming. <https://learnopengl.com/>

OpenGLBook.com. (s.f.). Introduction to Modern OpenGL. <http://www.openglbook.com/>

Song, H. (s.f.). OpenGL Projection Matrix. Recuperado de http://www.songho.ca/opengl/gl_projectionmatrix.html